



PDF Download
3717061.pdf
24 December 2025
Total Citations: 7
Total Downloads:
1789

Latest updates: <https://dl.acm.org/doi/10.1145/3717061>

RESEARCH-ARTICLE

An Empirical Study of Retrieval-Augmented Code Generation: Challenges and Opportunities

ZEZHOU YANG, Harbin Institute of Technology, Harbin, Heilongjiang, China

SIRONG CHEN, Harbin Institute of Technology, Harbin, Heilongjiang, China

CUIYUN GAO, Harbin Institute of Technology, Harbin, Heilongjiang, China

ZHENHAO LI, Concordia University, Montreal, QC, Canada

XING HU, Zhejiang University, Hangzhou, Zhejiang, China

KUI LIU, Huawei Technologies Co., Ltd., Shenzhen, Guangdong, China

[View all](#)

Open Access Support provided by:

[Harbin Institute of Technology](#)

[Concordia University](#)

[Zhejiang University](#)

[Huawei Technologies Co., Ltd.](#)

Published: 14 August 2025
Online AM: 14 February 2025
Accepted: 01 January 2025
Revised: 09 October 2024
Received: 18 November 2023

[Citation in BibTeX format](#)

An Empirical Study of Retrieval-Augmented Code Generation: Challenges and Opportunities

ZEZHOU YANG, SIRONG CHEN, and CUIYUN GAO, Harbin Institute of Technology, Shenzhen, China

ZHENHAO LI, Concordia University, Montreal, Quebec, Canada

XING HU, Zhejiang University, Hangzhou, China

KUI LIU, Huawei Technologies Co., Ltd., Shenzhen, China

XIN XIA, Zhejiang University, Hangzhou, China

Code generation aims to automatically generate code snippets of specific programming language according to natural language descriptions. The continuous advancements in deep learning, particularly pre-trained models, have empowered the code generation task to achieve remarkable performance. One main challenge of pre-trained models for code generation is the semantic gap between developers' natural language requirements and source code. To address the issue, prior studies typically adopt a retrieval-augmented framework for the task, where the similar code snippets collected by a retrieval process can be leveraged to help understand the requirements and provide guidance for the generation process. In a retrieval-augmented framework, similar data can be retrieved from the database using a retrieval algorithm, and original input data can be fused with retrieved data by different fusion strategies. However, there is a lack of systematic study on the application of this framework for code generation, including the impact of the final generated results and the specific usage of the framework. In this article, we choose three popular pre-trained code models, namely CodeGen, UniXcoder, and CodeT5, to assess the impact of the quality and utilization of retrieved code on the retrieval-augmented framework. Our analysis shows that the retrieval-augmented framework is beneficial for improving the performance of the existing pre-trained models. We also provide suggestions on the utilization of the retrieval-augmented code generation framework: BM25 and Sequential Integration Fusion are recommended due to their convenience and superior performance. Sketch Filling Fusion, which extracts a sketch of relevant code, could help the model improve its performance further. Additionally, we conduct experiments to investigate the influence of the retrieval-augmented framework on large language models for code generation, showing the effectiveness of the framework, and we discuss the tradeoff between performance improvement and computational costs in each phase within the framework.

This research is supported by National Natural Science Foundation of China (Project No. 62472126), Natural Science Foundation of Guangdong Province (Project No. 2023A1515011959), Shenzhen-Hong Kong Jointly Funded Project (Category A, No. SGDX20230116091246007), Shenzhen Basic Research (General Project No. JCYJ20220531095214031), Shenzhen International Science and Technology Cooperation (Project No. GJHZ20220913143008015), the Major Key Project of PCL (Grant No. PCL2024A05), and CCF-Huawei Populus Grove Fund.

Authors' Contact Information: Zezhou Yang, Computer Science, Harbin Institute of Technology, Shenzhen, China; e-mail: yangzezhou@stu.hit.edu.cn; Sirong Chen, Computer Science, Harbin Institute of Technology, Shenzhen, China; e-mail: 23S051017@stu.hit.edu.cn; Cuiyun Gao (corresponding author), Computer Science, Harbin Institute of Technology, Shenzhen, China; e-mail: gaocuiyun@hit.edu.cn; Zhenhao Li, Concordia University, Montreal, Quebec, Canada; e-mail: zhenhao.li@ieee.org; Xing Hu, Zhejiang University, Hangzhou, China; e-mail: xinghu@zju.edu.cn; Kui Liu, Huawei Technologies Co., Ltd., Shenzhen, China; e-mail: brucekui.liu@gmail.com; Xin Xia, Zhejiang University, Hangzhou, China; e-mail: xin.xia@acm.org.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2025/8-ART188

<https://doi.org/10.1145/3717061>

CCS Concepts: • **Software and its engineering** → **Software creation and management**; **Software development techniques**;

Additional Key Words and Phrases: code generation, retrieval-augmented methods, empirical study

ACM Reference format:

Ze Zhou Yang, Sirong Chen, Cuiyun Gao, Zhenhao Li, Xing Hu, Kui Liu, and Xin Xia. 2025. An Empirical Study of Retrieval-Augmented Code Generation: Challenges and Opportunities. *ACM Trans. Softw. Eng. Methodol.* 34, 7, Article 188 (August 2025), 28 pages.
<https://doi.org/10.1145/3717061>

1 Introduction

With the development of deep learning, pre-trained models have demonstrated remarkable performance in various code intelligence tasks. These models are pre-trained on large-scale datasets including both code and text, and subsequently fine-tuned for specific downstream tasks [16, 21, 48, 68]. The utilization of pre-trained models facilitates the effective resolution of a multitude of challenging tasks that are previously considered difficult. Code generation task, which aims at automatically generating code based on natural language descriptions, is improved continuously by pre-trained models. Prior researches predominantly employ Seq2Seq models to perform code generation tasks [32, 45], some of which are augmented with structural information to bolster the syntactic correctness of the generated code [31, 55, 74]. These models can generate functional code on some simple datasets. To handle the intricate development environments, researchers have introduced pre-trained models for code, which can achieve superior experimental results for code generation compared to previous models that are not pre-trained [21, 48, 68]. The task has proven effective in improving the efficiency of daily software development [9, 11, 12, 62]. Consequently, the task has attracted considerable attention among industry and academia, encouraging numerous practitioners and researchers to undertake a series of comprehensive studies [9, 11, 12, 44, 50, 67].

Recently, there have been studies proposing retrieval-augmented approaches to generate more accurate programs for the code generation task [25, 41, 54, 77]. The relevant code snippets retrieved from a retrieval database are explicitly referenced as guideline within a code generation model to enhance the generation performance [54] and to improve the informativeness of the generated code [14]. Despite the effectiveness, these retrieval-augmented approaches have not been adopted as a universal framework to help the existing models generate better code. One main reason is that there is a lack of a comprehensive exploration on the utilization of the **Retrieval-Augmented Framework (RAF)** for code generation. For example, noisy retrieved code snippets could decrease the model performance. In the RAF for code generation, relevant code snippets can be retrieved as reference by different retrieval techniques, and the original input can be augmented with the reference by various fusion strategies. Both the retrieval and fusion procedures can impact the model performance. It is necessary yet under-explored whether different code pre-trained models can benefit from the RAF for code generation. The impact of retrieval techniques and fusion strategies on the model performance is also worth investigating for achieving in-depth insights for researchers and practitioners.

To comprehensively explore the utilization of the RAF for code generation, we conduct extensive experiments in the article. Three popular pre-trained code models (i.e., CodeGen [50], UniXcoder [21] and CodeT5 [68]) are evaluated on three widely used datasets (i.e., CONCODE [32], CoNaLa [72], and HearthStone [45]). Specifically, we aim at answering the following three **Research Questions (RQs)**:

RQ1: What is the impact of RAF on the performance of various pre-trained models for code generation task? In this RQ, we aim at investigating the impact of the RAF on the model performance and its generalization. Without loss of generality, the basic text retrieval technique, BM25 [64], is adopted to retrieve relevant code snippets. These retrieved code snippets are concatenated with the original natural language description directly, serving as augmented data for the model. We fine-tune the pre-trained code models with the augmented data and demonstrate that RAF is universally applicable to the pre-trained models on different datasets for code generation. Experiment results show that the code generation performance of all the three models can benefit from the framework in various metrics. Especially on the HearthStone dataset, the three models exhibit an average improvement of 41.60% in the EM metric, 9.01% in the BLEU metric, and 8.69% in the CodeBLEU metric.

RQ2: How do the retrieval techniques affect RAF for code generation? In this RQ, we explore how different kinds of retrieval techniques (i.e., code search models and text retrieval algorithms) in the RAF affect model performance. The process that retrieves code snippets according to natural language can be regarded as code search task, so code search models can be used as retrieval techniques directly. In this article, we choose CodeBERT [16] and CoCoSoDa [60] as code search models to retrieve similar code snippets from training set based on the input natural language descriptions. Popular text retrieval techniques such as BM25 [64] and RetroMAE [71] are also involved. Our experiment results show that all three models achieved the highest improvement from the retrieved results of BM25 on CONCODE and HearthStone. On CoNaLa, BM25 shows optimal performance for CodeT5 and suboptimal for CodeGen, which further demonstrates the effectiveness of BM25 for the RAF.

RQ3: What is the impact of different strategies for fusing the retrieved results on the model performance? In this RQ, we study how to better integrate the retrieved code snippets for the code generation task from two aspects, including the number of retrieved results and the fusion strategies. For the fusion strategies, besides sequentially integrating the retrieved results with the natural language description, we also consider the **Sample Expansion Fusion (SEF)**, **Vectorized Decoding Fusion (VDF)** [33], and **Sketch Filling Fusion (SFF)** [41]. Our experiment results show that the quantity of retrieved code snippets should be determined according to the attributes of the specific dataset such as input/output length. SEF with retrieved code snippets can improve the model performance to a great extent. Based SEF, SFF yields an average improvement of 14.83% in the BLEU metric and 8.05% in the CodeBLEU metric across the three datasets for original CodeT5. This improvement stands out as the highest among the four fusion strategies, suggesting that constructing sketches of relevant code could further enhance the model. However, the training associated with SFF is notably resource-intensive. In terms of balancing computational cost and performance enhancement, **Sequential Integration Fusion (SIF)** proves to be a more cost-effective approach.

Through the large-scale empirical study, we achieve some findings and summarize the key findings as below.

- (1) RAF could be adopted to improve the performance of various pre-trained models for code generation.
- (2) More complex retrieval techniques do not necessarily lead to better code generation results. BM25 is proven to be the most effective retrieval technique for code generation.
- (3) SIF with retrieved results is a simple but effective fusion strategy. Despite the high computational costs, SFF can further improve the model performance.

The major contributions of this article are as follows:

- (1) This article serves as the first empirical study on the performance of RAF for code generation.

- (2) We explore how different retrieval techniques and fusion strategies affect the performance of RAF for code generation.
- (3) We discuss the implications of our findings and provide actionable insights on the specific usage of the framework.

The rest of the article is organized as follows: Section 2 briefly introduces background and related work. Section 3 elaborates the RAF. Section 4 introduces the setup of our experiment study. Section 5 presents the experiment results. Section 6 discusses the implications of findings and threats to validity. Section 7 presents the conclusions of the article.

2 Background and Related Work

2.1 Pre-Trained Models in Code Intelligence

Pre-trained models accumulate knowledge from large-scale unlabeled data through self-supervised training strategies, exhibiting superior generalization. Subsequent fine-tuning can yield commendable results on multiple tasks [56]. Encoder-only models are consistently used for code comprehension tasks with bidirectional attention. CodeBERT[16] is pre-trained on NL-PL pairs in six programming languages with Masked Language Modeling and Replace Token Detection. GraphCodeBERT [22] designs pre-training tasks about dataflow to improve code representation. Decoder-only models are good at auto-regressive tasks like code generation and code completion with unidirectional attention. CodeGPT [48] is pre-trained on Java and Python corpus sourced from CodeSearchNet [30], which has the same architecture and training objective as GPT-2 [56]. Incoder [17], employing a decoder-only architecture, demonstrates an impressive ability to predict tokens not only in a left-to-right sequence but also within the middle, utilizing information from both ends. CodeGEN [50] takes a conversational approach to program generation, where the process of code generation is described as multiple rounds of dialogue between the user and the system. The step-by-step approach can break down long and complex intents into multiple simple intents, reducing the search space for models in each round of conversation.

In addition to employing either the encoder or the decoder of the Transformer [65] independently, there are notable works that utilize the whole structure of Transformer in code intelligence, with UniXcoder [21] and CodeT5 [68] standing out as popular models. UniXcoder, as a unified pre-trained model that incorporates semantic and syntax information from code comment and **Abstract Syntax Tree (AST)**, can utilize mask attention matrices with prefix adapters to enable switching between Encoder-only, Decoder-only and Encoder-Decoder architecture. CodeT5 is a unified pre-trained encoder-decoder model that leverages the token type information from code and has the ability to seamlessly support both code understanding and generation tasks. These end-to-end architectures have been proved well-suited for various code-related tasks, including bug fixing, code translation, code summarization, and code generation [51].

2.2 Code Generation

In recent years, code generation, a burgeoning research topic, has attracted widespread attention. Ling et al. [45] design a neural network architecture to convert the natural language description of a card into specific code implementation in Hearthstone or Magic the Gathering. Yin et al. [73] propose a novel approach powered by a grammar model which first translates natural language into an AST and subsequently regenerates it into code in a high-level programming language. On this basis, many methods [31, 55, 61, 74] also leverage the AST of code to facilitate code generation in high-level programming languages. Pre-trained models using code corpus have achieved excellent results in code generation, such as CodeGPT [48], PLBART [1], UniXcoder [21], and CodeT5 [68].

The practice of pre-training followed by fine-tuning has become a mainstream pipeline to code generation.

For the code generation model, the decoding space of the model is large, which may lead to the poor quality of the generated code [9]. To alleviate such a problem, some work [25, 26, 41, 54, 77] incorporates external knowledge to enhance the code generation model by means of retrieval. REDCODER [54] enhances code generation task by retrieving similar code snippets and enhances code summarization task by retrieving relevant text. Hashimoto et al. [25] design a retrieve-and-edit framework, which retrieves the code first and then edits the retrieved results as the output code. SKCODER [41] extracts templates from retrieved results as sketch and edits the sketch into the output code. DocPrompting [77] maintains a document library that stores the specification documents of the code and enhances the code generation model by retrieving the document library. In addition to enhancing generation through retrieval, some work [7, 38, 66] reconstructs the input and output of the model via pre-processing and post-processing techniques, resulting in higher-quality code. Specially, CodeT [7] allows for the selection of the optimal result within the candidate set of generated code by automatically generated test cases. CodeRL [38] determines the performance of the generated code through test cases and uses this to supervise the training of the model. COMPCODER [66] incorporates both a generator and a discriminator to enhance the model's compilation capabilities.

In the early works, semantic parsing datasets [13, 75] are commonly employed for evaluating the performance of code generation models, where HearthStone [45] is widely used due to its clear code implementation and structured requirement description. Subsequently, the datasets specifically designed to mirror real-world programming scenarios come into prominence [32, 45, 72]. For instance, CoNaLa, a dataset sourced from Stack Overflow, is meticulously constructed to serve as a benchmark for tasks that necessitate the generation of code from natural language descriptions. Currently, there is a wealth of diverse datasets in the field of code generation, with a primary emphasis on Python and Java. CONCODE [32] that is selected in CodeXGLUE [48] has been a comprehensive and necessary evaluation dataset for code generation. Furthermore, with the advancements in generative models, test case datasets [3, 5, 9, 28] have emerged, further assessing the code generation capabilities of models based on the pass@k metric [9]. Simultaneously, there is a discernible shift from monolingual datasets towards multilingual ones [5, 69, 78], indicating a prevailing trend.

Due to the economic and technical potential of code generation, corporations continue to roll out pre-trained models for code generation [9, 11, 44], and it can be found that the number of parameters and performance of these models continue to exceed conventional understanding. When **Large Language Models (LLMs)** demonstrate excellent code generation capabilities, code generation tasks have received more widespread attention and heated discussions. Several enterprises and institutions have launched a number of LLMs for code generation including CodeGeeX [12], AlphaCode [44], PanGu-Coder [11], and CodeX [9]. With the continuous increase in the number of model parameters, the application ways are gradually diversified. In particular, ChatGPT [6] and GPT-4 [52] have strong generation capabilities in various fields, including code generation. Many larger models of code are optimized by instruction-tuning in training stage [59, 67]. In inference stage, in-context learning and chain-of-thought prompts are used to improve the generated results [4, 34, 40, 42].

2.3 Retrieval-Augmented Generation

Retrieval-augmented generation refers to improving the generation performance with the retrieved results provided by retrieval techniques. For language models, the knowledge learned from training data is all stored in the parameters of the neural network. The model might encounter challenges in

generating the correct answer due to numerous parameters [9]. Furthermore, when confronted with knowledge that has never been learned during pre-training, the model might fail to provide the accurate response. The retrieved results can be regarded as a supplement to the implicit knowledge stored in the parameters of language models, encouraging the model to produce more accurate outputs [24]. In addition, database can be modified and constantly updated, enabling the trained models to adapt to a broader range of new data [77]. In other words, retrieval-augmented generation achieves scalability of the modification or replacement of retrieval sources without the need to alter the models.

The **k-Nearest Neighbor Language Model (kNN-LM)** [36] retrieve the k most similar training contexts for test context according to the distance in the embedding space of the pre-trained language model. In fact, the k training contexts correspond to k training targets. By normalizing and aggregating k training targets, kNN-LM can get a target distribution from kNNs, and the pre-trained language model can generate another target distribution directly according to current input. kNN-LM can merge the two distributions above by weighted sum to get the final target distribution. Different from kNN-LM, **Retrieval-Augmented Language Model (REALM)** [24], whose workflow can be summarized as the retriever-and-reader, has two components that are both trained. One is a neural knowledge retriever, which retrieves similar text with input using a dense inner product model. The other is the knowledge-augmented encoder, which predicts the final results based on input and the retrieved text in the last step. Actually, the prediction cannot generate texts using the encoder but extract a contiguous sequence from the retrieved text as the result. A similar workflow has been proposed and developed [8, 49] before REALM occurs. To give more powerful ability to process different kinds of tasks, Lewis et al. [39] replace reader with generator under the workflow of retriever-and-reader, which is also called retriever-and-generator. Indeed, retrieval-augmented methods have been used in code generation and have developed for a long time. Hashimoto et al. [25] proposes a retrieve-and-edit framework that edits the retrieved results to the desired output instead of generating code directly. The workflow of retrieve-and-edit is similar to retriever-and-generator [39], but the details of the two models are different. In specific, retrieval has more powerful components and can combine retriever with generator to fine-tune end-to-end.

Although the development of the models is rapid, fine-tuning following pre-training remains an indispensable paradigm for small-sized models. Based on this mode, the utilization of similar retrieved results to improve pre-trained models for code generation has demonstrated effectiveness, attracting considerable attention. However, most of these approaches only use single configuration, focusing on a specific view, and do not systematically summarize the RAF and its usage from various aspects. Therefore, we abstract three phases of the RAF for code generation (retrieval phase, fusion phase, and generation phase) and conduct a systematic study. For the retrieval phase, previous studies related to code search highlight that deep learning-based models can mitigate the semantic gap between query and code compared with statistics-based algorithms [46]. However, deep learning-based techniques rely on labeled data to learn the parameters of the models, which might bring additional training costs [76]. These models may perform worse than statistics-based approaches under zero-shot settings [63]. For the fusion phase, while many fusion strategies can be used in the RAF from previous studies [33, 41], there is a lack of systematic exploration into the effectiveness and impact of the fusion strategies. For the generation phase, our main concern is the impact of the RAF on various pre-trained code generation models.

Recently, LLMs have shown impressive results in various downstream tasks across domains [9, 12, 19]. However, they encounter challenges in real-world scenarios, such as hallucinations [29], outdated knowledge [20], and unclear reasoning processes [37]. Retrieval-augmented generation can prompt LLMs to produce reliable and accurate results by integrating real-time factual knowledge

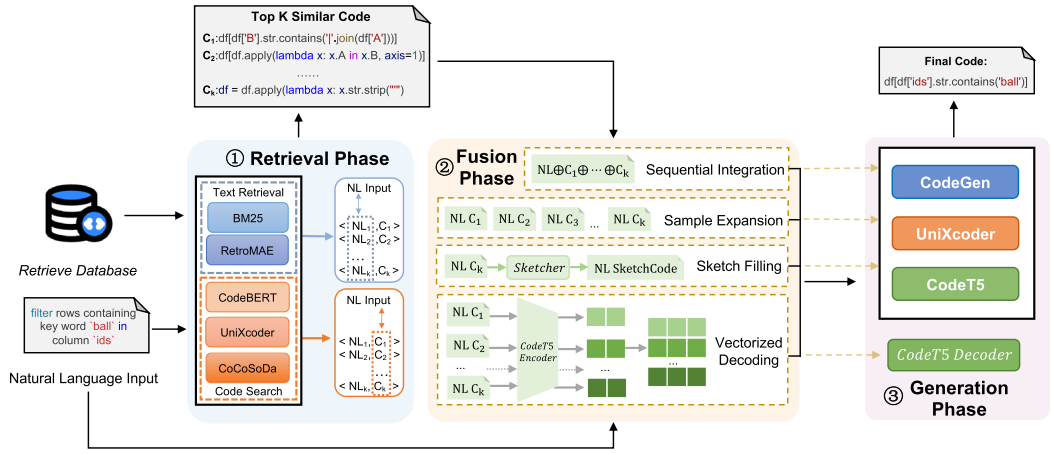


Fig. 1. Overview of RAF for code generation.

retrieved from external databases. Considering its effectiveness for LLMs on other tasks, we additionally conduct experiments to explore its impact on LLMs for code generation, discussed in detail in Section 6.

3 RAF

3.1 Overview

Figure 1 illustrates the overview of the general RAF for code generation. As can be seen, given a natural language input, the framework mainly includes three phases, i.e., *Retrieval Phase*, *Fusion Phase*, and *Generation Phase*. The details of each phase are as below. ① *Retrieval Phase*: The phase aims at retrieving relevant code snippets based on the provided natural language description; ② *Fusion Phase*: In the phase, the retrieved code snippets are integrated with original natural language description as augmented input for the generation phase. ③ *Generation Phase*: The phase leverages the augmented input to generate the code. All the phases can contribute to the model performance. Specifically, we first study the effectiveness and generalizability of the RAF by adopting various pre-trained code models in the *Generation Phase*. We then explore the impact of different retrieval techniques in the *Retrieval Phase* and fusion strategies in the *Fusion Phase* on the model performance.

3.2 Retrieval Phase

In Retrieval Phase, the source of the retrieval database can be collected from GitHub, Stack Overflow, existing datasets, and so on. The typical format of the retrieval database is <Natural Language Description, Code Snippet>. There are two distinct ways to obtain the final retrieved code snippets. The first way is regarding the database as a dictionary, where Natural Language Descriptions serve as the keys and corresponding Code Snippets act as the values. The text retrieval algorithms in **Natural Language Processing (NLP)** can measure the similarity between current natural language input and natural language description in database, and then lookup the corresponding code snippets as the final retrieved results. From the other perspective, retrieving the relevant code snippets in database according to the natural language input meets the definition of code search. This means that various code search models can be used to retrieve relevant code snippets according to the natural language input. In detail, code search models can directly measure the similarity between natural language input and code snippets in database and return similar code snippets as retrieved results. In this article, we select five retrieval algorithms including two text retrieval

algorithms (i.e., BM25 [64] and RetroMAE [71]) and three code search models (i.e., CodeBERT [16], UniXcoder [21], and CoCoSoDa [60]).

3.2.1 Text Retrieval Algorithms. There are two text retrieval algorithms in our experiments. They retrieve similar natural language descriptions with natural language input and select the corresponding code snippets as the retrieved results.

BM25 [64] is commonly utilized for assessing the correlation between search terms and documents in document set. Given a search term *Text* and a document set D , $d \in D$ represents a document within the document set. The document set D serves as the search database, and within this database, the natural language in <Natural Language, Code Snippet> example pairs constitutes a document. During the correlation calculation process, the search term *Text*, serving as natural language input, is initially divided into multiple tokens $\{t_1, t_2, \dots, t_i, \dots, t_n\}$. The correlation score can be calculated as:

$$Score(T, d) = \sum_{i=1}^n (w_i R(t_i, d)), \quad (1)$$

where w_i donates the weight of t_i , which is used to assess the importance of t_i for document set D . $R(t_i, d)$ indicates the degree of correlation between t_i and the document d . w_i is calculated as follows:

$$w_i = IDF_i = \log \frac{N - df_i + 0.5}{df_i + 0.5}, \quad (2)$$

where N donates the number of documents in the database and df_i donates the number of documents containing t_i . $R(t_i, d)$ can be calculated according to the following formula:

$$R(t_i, d) = \frac{f_i * (k + 1)}{f_i + k * (1 - b + b * \frac{L_d}{L_{avg}})}, \quad (3)$$

where f_i donates the frequency value of t_i in document d , L_d donates the length of the document d , and L_{avg} donates the average length of all documents. In the above formula, b and k are trainable parameters.

RetroMAE [71] is a pre-trained text retrieval model based on **Masked Auto-Encoder (MAE)** with asymmetric masking ratios for encoder and decoder. It includes a full-scale BERT as encoder and a single-layer Transformer Decoder as decoder. With the workflow of MAE, the input sentences are polluted with different masks and encoded as masked sentence embedding by encoder at first. Subsequently, the decoder can recover the original sentence according to the masked sentence embedding. Based on MAE, RetroMAE proposes an enhanced decoding process that can capture more training signals and diversified contexts with two-stream self-attention and position-specific attention mask. In our experimental setup, we directly employ RetroMAE for inference because of the absence of labeled natural language data to train it for text retrieval tasks.

3.2.2 Code Search Models. There are three code search models in our experiments. They can search the similar code snippets directly according to the natural language input. To better adapt to the data distribution in the code generation domain, all three models are trained on the code search task using the training set of the corresponding code generation dataset.

CodeBERT [16] is the first bimodal (i.e., natural language and programming language) pre-trained model for multiple programming languages with the same architecture as RoBERTa-base [47]. For code search task, the natural language input and the code snippets are concatenated with “[CLS]” as the input of CodeBERT. The representation of “[CLS]” is processed through a softmax layer to measure the semantic relevance between code and natural language input.

UniXcoder [21] is a unified pre-trained code model that is compatible with three different architectures (i.e., Encoder-Decoder model, Encoder-only model, and Decoder-only model) and controls the operation by mask attention matrices with prefix adapters. To acquire semantic embedding of code, UniXcoder introduces two novel pre-training tasks: Multi-Modal Contrastive Learning and Cross-Modal Generation. Both of them can enhance the understanding ability of UniXcoder and improve the performance in downstream tasks. For code search tasks, UniXcoder can utilize the encoder-only mode to separately encode natural language and code, followed by similarity computation.

CoCoSoDa [60] leverages contrastive learning for code search task and has achieved state-of-the-art performance up to now. In detail, CoCoSoDa introduces four soft data augmentation and incorporates the momentum contrastive learning (MoCo) framework [27] to learn the representation of code and natural language query. The core idea of contrastive learning is pulling similar representations and pushing apart discrepant representations. Ultimately, both code and natural language query can be encoded into the high-dimensional vectors by code and query encoders designed using the same architecture as UniXcoder [21], and their similarity can be measured with cosine similarity.

3.3 Fusion Phase

Fusion Phase aims to connect the Retrieval phase and the Generation Phase. If the code snippets retrieved by retrieval techniques are seen as the references that can provide additional knowledge, the fusion strategy in Fusion Phase is the guidance of the references for the subsequent code generation. The fusion strategy utilizes the retrieved code snippets to alter the input while keeping the model architecture unchanged. Here, we offer a brief introduction of four fusion strategies employed in Fusion Phase as follows:

SIF. SIF is a naive strategy to fuse retrieved code and natural language input. The k retrieved code snippets are seamlessly concatenated following the original natural language input by means of a special token `<retrieved_code>` as new input.

SEF. SEF refers to a strategy of enriching the original training dataset. The k retrieved code snippets can be individually concatenated with their corresponding natural language input as k new instances, while maintaining the unchanged target code. In other words, the training data can be expanded by creating new instances based on the retrieved results. The volume of training data, except for the original data, is increased by a factor of k . In the Generation Phase, the model can generate code according to both the original natural language input and the most similar code snippet.

VDF. Based on SEF that can transform k retrieved code snippets into k new instances, VDF encodes each new instance into a vector. The k vectors with the same natural language input can be concatenated along the hidden dimension and utilized as input to the decoder, resulting in the final generation output. In other words, VDF changes the fusion process by encoding code snippets into vectors, which can solve the problem of truncating and keep the information aggregated [33]. In our experiment, we use the encoder of CodeT5 to obtain the representation of new instances and fuse them in the decoder of CodeT5.

SFF. VDF can be considered as utilizing an encoder to extract the semantic information of the retrieved code snippets in a high-dimensional space. However, VDF cannot explicitly capture similar structures in similar code. Extracting the sketch of the most similar code snippet is advantageous for filtering out potentially irrelevant details and preserving the most useful and pertinent information. The sketch can also be seen as a template, explicitly offering additional structure information to the model. In specific, we use a neural encoder and a linear classification layer following SKCODER [41] to finish the sketch extraction of relevant code.

In addition to the specific fusion strategy, it should be considered how the number of code snippets used in the Fusion Phase impacts the model performance, which will be discussed in 5.4.

3.4 Generation Phase

In the Generation Phase, the generative model aims at generating the final code based on the data constructed during the Fusion Phase. The use of retrieved code snippets related to natural language input varies with the fusion strategy, while the architecture of the models remains unchanged. The original generative model presents the code generation tasks in the following format:

$$F(<x_1, x_2, \dots, x_n>) = <y_1, y_2, \dots, y_m>, \quad (4)$$

where $<x_1, x_2, \dots, x_n>$ is the natural language input and $<y_1, y_2, \dots, y_m>$ is the target code snippets. F represents the process functionality of generative models, which can achieve the mapping from the natural language input into generated code snippets.

Code Generation with SIF. SIF is used to obtain an augmented dataset by appending similar retrieved code snippets after the original natural language input. It is necessary to introduce special separating tokens to help the generative model distinguish between the original input and the reference code snippets, thereby shifting the code generation process:

$$F(<x_1, x_2, \dots, x_n>, [\text{retrieved_code}], C_1, [\text{retrieved_code}], \dots, C_k) = <y_1, y_2, \dots, y_m> \quad (5)$$

where $[\text{retrieved_code}]$ denotes the special separating tokens, and C_k represents to the k th retrieved similar code snippets.

Code Generation with SEF. Contrasting with SIF, SEF individually elaborates on each of the k retrieved code snippets after the original natural language input. Consequently, it is employed in the following manner for k retrieved code snippets during the training stage:

$$\begin{cases} F(<x_1, x_2, \dots, x_n>, [\text{retrieved_code}], C_1) &= <y_1, y_2, \dots, y_m> \\ &\dots \\ F(<x_1, x_2, \dots, x_n>, [\text{retrieved_code}], C_k) &= <y_1, y_2, \dots, y_m> \end{cases} \quad (6)$$

During the inference stage, the model generation process is subsequently represented as:

$$<\hat{y}_1, \hat{y}_2, \dots, \hat{y}_m> = F(<x_1, x_2, \dots, x_n>, [\text{retrieved_code}], C), \quad (7)$$

where C denotes the most similar code snippets retrieved from the codebase, and the models can generate the code prediction $<\hat{y}_1, \hat{y}_2, \dots, \hat{y}_m>$ based on the natural language input and C .

Code Generation with VDF. VDF involves encoding each instance from the SEF into a vector. These vectors with the same natural language input are concatenated and then fed into the decoder. This fusion process necessitates both an encoder and a decoder:

$$\begin{cases} v_{C_1} &= \text{Encoder}(<x_1, x_2, \dots, x_n>, [\text{retrieved_code}], C_1) \\ &\dots \\ v_{C_k} &= \text{Encoder}(<x_1, x_2, \dots, x_n>, [\text{retrieved_code}], C_k) \end{cases} \quad (8)$$

$$\text{Vector}_{\text{Input}} = \text{concat}([v_{C_1}, \dots, v_{C_k}]) \quad (9)$$

$$\text{Decoder}(\text{Vector}_{\text{Input}}) = <y_1, y_2, \dots, y_m> \quad (10)$$

Table 1. Statistics of the Datasets for Code Generation

Dataset	Train/Test/Validation	Programming Language	Max-Avg-Min length of Input/Output
CONCODE	100,000/2,000/2,000	Java	2,246/264-213/33-18/6
CoNaLa	2,179/500/200	Python	62/84-16/16-1/1
HearthStone	533/66/66	Python	115/636-74/131-54/78

Code Generation with SFF. SFF extracts the sketch of the retrieved similar code snippets, necessitating that the model edits the sketch to obtain the final code:

$$\begin{cases} F(\langle x_1, x_2, \dots, x_n \rangle, [\text{code_sketch}], S_1) = \langle y_1, y_2, \dots, y_m \rangle \\ \dots \\ F(\langle x_1, x_2, \dots, x_n \rangle, [\text{code_sketch}], S_k) = \langle y_1, y_2, \dots, y_m \rangle, \end{cases} \quad (11)$$

where $[\text{code_sketch}]$ denotes the special separating tokens, and S_k represents to the sketch of k th retrieved similar code snippets. The inference stage is similar to SEF:

$$\langle \hat{y}_1, \hat{y}_2, \dots, \hat{y}_m \rangle = F(\langle x_1, x_2, \dots, x_n \rangle, [\text{retrieved_code}], S) \quad (12)$$

where S denotes the sketch of the most similar code snippets retrieved from the codebase.

4 Experiment Study Setup

4.1 Dataset

We evaluate the RAF for code generation on three well-recognized datasets: CONCODE [32], CoNaLa [72], and HearthStone [45]. The statistics of these three datasets are summarized in Table 1.

CONCODE [32] in CodeXGLUE [48] is one of the most popular datasets for code generation task. It includes about 33,000 Java projects collected on GitHub. According to the GitHub repository, CONCODE can be divided into 100,000 instances for training and 4,000 instances for validation and testing. The repository-based partitioning keeps the domain in the test set separate from the training set, which helps test the real generalization of models for unseen natural language descriptions. Each instance is a tuple of natural language descriptions, code environments, and code snippets, where the code environment includes other member variables and member functions in the class. *CoNaLa* [72] comprises 2,879 manually annotated questions along with their corresponding Python solution instances from Stack Overflow. These instances encompass genuine natural language queries posed by programmers with diverse intentions. The length of input and output are shorter compared to the other two datasets as shown in Table 1.

HearthStone [45] is a collection of Python classes implemented for the HearthStone card game, containing 665 different HearthStone cards. Each card contains a set of fields delineating the card information and Python code snippets implementing its corresponding functions. These fields include semi-structured descriptions such as the card name, cost, attack, description, and other attributes. Since most of the fields are similar among the cards, the code structures of different cards are comparable.

We split the CONCODE and HearthStone datasets into training, test, and validation sets by following the original papers. Although CoNaLa has already been split into 2,379 training and 500 test instances, there are no validation instances available for experiments. To facilitate our

Table 2. Overview of the Pre-Trained Code Models in the Generation Phase

Model	Parameters	Pre-Trained Data	Input Length	Output Length
CodeGen	350M	The Pile, BigQuery, BigPython	2,048	-
UniXcoder	126M	CodeSearchNet	350	150
CodeT5	223M	CodeSearchNet	512	512

experimentation, we select 200 instances randomly from training data as the validation set for CoNaLa.

4.2 Pre-Trained Code Models

In terms of the model architecture, pre-trained models in code intelligence can be categorized into three types: Encoder-Decoder, Encoder-only, and Decoder-only. The models with the architecture of Encoder-only cannot finish generation tasks due to their inherent bidirectional representation. In our experiments, three existing pre-trained code models are chosen to verify the effectiveness of the RAF. These three models are CodeGen [50], UniXcoder [21], and CodeT5 [68]. In the three models, CodeGen is a Decoder-only model, and the other two models (i.e., UniXcoder and CodeT5) have both Encoder-Decoder architecture. They stand for different training strategies and generation processes, and the details are introduced as below. Table 2 presents the overview of these three pre-trained code models.

CodeGen [50], a kind of auto-regressive transformer, has a similar architecture to GPT-NEO [18]. The training objective is to maximize the likelihood of the target sequence given the context with a natural language corpus and programming language data collected from GitHub. CodeGen comprises three versions according to sequential training datasets. In short, CodeGen-NL is trained on The Pile. CodeGen-MULTI continues training on BigPython based on CodeGen-NL, and CodeGen-MONO continues training on BigPython based on CodeGen-MULTI. In our experiment, we choose the last version (i.e., CodeGen-MONO) due to its superior performance.

UniXcoder [21] can utilize the encoder-decoder mode for code generation tasks. Other detailed information of UniXcoder has been described in Section 3.2.2.

CodeT5 [68] based on T5 [57], is a pre-trained model that can accomplish various code intelligence tasks through generation forms, much like how T5 is used for NLP tasks. CodeT5 fully considers code-specific sequence and structural information with Identifier-Aware Denoising Pre-training in pre-training stage. In the subsequent stage of pre-training, CodeT5 leverages NL-PL bimodal data for dual generation to close the gap between discrete knowledge from pre-training and continual knowledge from fine-tuning. In a recent empirical study [51], CodeT5 has been demonstrated as one of the most potent pre-trained models for code generation task.

4.3 Metrics

Exact Match Accuracy (EM) represents the percentage of exact matches between predicted code and reference code (i.e., ground-truth), which shows that this metric is the most restrictive. The metric is to measure the ability of the model to generate identical code, defined as

$$EM = \frac{\sum_{i=1}^{|D|} (y_i == \hat{y}_i)}{|D|}. \quad (13)$$

BLEU [53] is an important metric to evaluate the quality of machine translation, and it is also widely used in other generation tasks. BLEU compares the n-gram in the generated code to measure the similarity with the reference code, where n-gram refers to the consecutive n tokens in the

sentence. In the code generation task, n is typically taken as 4, and we take BLEU-4 as one of our metrics to evaluate the RAF. To ensure the fairness of evaluation, the calculation of BLEU metric needs to introduce a penalty term for n -gram operation. BLEU can then be expressed as the product of n -gram weighting and the penalty term. The calculation process is shown as follows:

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N \omega_n \log p_n \right) \quad (14)$$

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{1 - \frac{r}{c}} & \text{if } c \leq r \end{cases}, \quad (15)$$

where p_n means the modified n -gram precision and ω_n is the weight. BP represents the brevity penalty. c is the length of the output code, and r is the length of the reference code.

Edit Distance (ED) measures the syntactic similarity by the minimum number of single token edits required to transform predicted code snippets to target code snippets. The allowed edit operations include insertion, deletion, and substitution of tokens. Define $ED(i, j)$ as the edit distance between the first i tokens of y and the first j tokens of $\hat{y}$. The edit distance $ED(i, j)$ can be computed using the following recurrence relation:

$$ED(i, j) = \begin{cases} 0 & \text{if } i = 0 \text{ and } j = 0 \\ i & \text{if } j = 0 \\ j & \text{if } i = 0 \\ \min \begin{cases} ED(i-1, j) + 1 \\ ED(i, j-1) + 1 \\ ED(i-1, j-1) + \delta(y[i-1], \hat{y}[j-1]) \end{cases} & \text{otherwise} \end{cases}, \quad (16)$$

where $\delta(a, b)$ is defined as:

$$\delta(a, b) = \begin{cases} 0 & \text{if } a = b \\ 1 & \text{if } a \neq b \end{cases}. \quad (17)$$

$Similarity_{AST}$ refers to the syntactic AST matching score to evaluate the structural information between predicted code snippets and the target code snippets. This is the formula to compute AST similarity:

$$Similarity_{AST} = 1 - \frac{\text{Tree Edit Distance}(T, \hat{T})}{\max(\text{Size}(T), \text{Size}(\hat{T}))}, \quad (18)$$

where T and $\hat{T}$ represent the AST of predicted code snippets and the target code snippets separately. The computation of tree edit distance bears similarity to that of edit distance, with the primary distinction being that tree edit distance is calculated based on tree nodes rather than individual tokens.

$CodeBLEU$ [58] is an improved metric based on BLEU by considering syntax and semantic information about code. In specific, $CodeBLEU$ incorporates the advantages of n -gram matching in BLEU and is enhanced by code syntax with AST and code semantics with dataflow:

$$CodeBLEU = \alpha \cdot BLEU + \beta \cdot BLEU_{weight} + \gamma \cdot Similarity_{AST} + \delta \cdot Similarity_{DF}, \quad (19)$$

where $\alpha, \beta, \gamma, \delta$ are weight coefficients to control the percent of different metrics. In contrast to assigning the same weight to each token in the BLEU calculation, $BLEU_{weight}$ assigns distinct weights to different tokens to obtain the n -gram matching score. $Similarity_{AST}$ is syntactic AST similarity score, and $Similarity_{DF}$ is semantic data flow similarity score.

4.4 Implementation Details

All the pre-trained models and the corresponding tokenizer in our experiment are loaded from the official repository in Huggingface.¹ We adopt the PyTorch² framework to implement all the models and accomplish various tasks. In our experiment, all the hyper-parameter settings of pre-trained models are the same as the original corresponding papers. All the datasets are organized in the form of <Natural Language Description, Code Snippets> and stored in JSON files. Our computing devices are two Intel(R) Xeon(R) Platinum 8276 CPU @ 2.20GHz with 28 cores and two NVIDIA A100 (80G graphic memory in total).

The learning process of retrieval-augmented models aligns with the three phases illustrated in Figure 1. To assess the effectiveness of the RAF without loss of generality, we conduct a series of experiments with BM25 and SIF in RQ1. For RQ2, we leverage the five retrieval techniques outlined in Section 2.3 to retrieve the Top k code snippets similar to the natural language input during Retrieval Phase. Subsequently, in Fusion Phase, we employ SIF to construct augmented data for different retrieved results. In RQ3, we perform a controlled experiment to determine the optimal number of concatenated retrieved results, and then we employ the four different fusion strategies introduced in Section 3.3 in Fusion Phase.

5 Empirical Study Results

5.1 RQs

We aim to answer the following RQs:

- RQ1:* What is the impact of RAF on the performance of various pre-trained models for code generation task?
- RQ2:* How do the retrieval techniques affect RAF for code generation?
- RQ3:* What is the impact of different strategies for fusing the retrieved results on the model performance?

5.2 RQ1: Effectiveness of RAF

To broadly evaluate the way to leverage the RAF into existing pre-trained models, we study the effectiveness of the RAF by comparing the performance of various pre-trained models before and after integration with the retrieved results on three distinct datasets: CONCODE [32], CoNaLa [72] and HearthStone [45].

We present the comparison results in Table 3. Compared with fine-tuning using the original datasets, the RAF achieves an average improvement of 6.79%, 11.45%, 6.93%, and 8.72% on CONCODE; 3.74%, 18.42%, 15.51%, and 16.75% on CoNaLa; 41.60%, 9.01%, 11.25%, and 8.69% on HearthStone for EM, BLEU, Similarity_{AST} and CodeBLEU, respectively. We perform a statistical significance test (t -test), and the results show that the models with the RAF outperform the original models at the significance level at 0.05 (p-value 0.035), demonstrating the effectiveness of the RAF on code generation. From the perspective of pre-trained models, within the RAF, the BLEU metric of CodeGen, UniXcoder, and CodeT5 are all increased by 14.90%, 7.27%, and 16.71% on average across the three datasets, respectively. Above all, the experiment results indicate that the RAF can improve the performance of various models within the same datasets, as well as identical models across different datasets, showcasing its generalization. In addition, the general improvements of multiple metrics, especially BLEU and CodeBLEU, can reflect the framework can help the model focus on the semantic and structural information of generated code during the code generation process.

¹<https://huggingface.co/models>

²<https://pytorch.org/>

Table 3. Results Comparison between the Base Model and the Retrieval-Augmented Models

Model	CONCODE					CoNaLa					HearthStone				
	EM	BLEU	ED	<i>Sim_{AST}</i>	CodeBLEU	EM	BLEU	ED	<i>Sim_{AST}</i>	CodeBLEU	EM	BLEU	ED	<i>Sim_{AST}</i>	CodeBLEU
CodeGen	19.55	21.83	19.13	36.80	27.33	9.00	12.74	9.48	22.15	17.30	10.61	50.35	23.81	58.85	40.76
+BM25	21.20	24.99	18.73	39.41	30.04	8.60	15.58	9.28	27.41	22.24	15.15	54.35	21.80	64.70	44.00
UniXcoder	22.80	32.42	18.74	41.11	35.73	10.60	12.76	9.33	24.36	20.18	13.64	58.60	19.38	57.14	45.70
+BM25	23.50	35.80	18.58	45.83	38.57	11.40	13.69	9.05	25.46	20.60	22.73	61.00	16.90	61.92	48.53
CodeT5	21.45	36.93	23.12	46.64	40.15	7.40	10.55	11.87	19.69	18.80	19.70	55.96	19.76	55.14	44.49
+BM25	23.35	40.42	21.91	47.67	43.48	8.00	13.26	10.46	23.29	22.49	22.73	64.35	14.95	63.65	49.80

The base models are fine-tuned using the original datasets. The retrieval-augmented models are fine-tuned using the datasets augmented by SIF with the code snippets retrieved by BM25. Under each metric the best performance is marked as bold.

Specifically, on the HearthStone dataset, the RAF yields notable enhancements in CodeGen by 42.79%, UniXcoder by 66.64%, and CodeT5 by 15.38% in the EM metric. The other four metrics exhibit the most substantial improvements among the three datasets. The phenomena may be attributed to the regular code structure of HearthStone. With the similar field of cards, the code snippets have a similar form so that the relevant retrieved results can provide the field of a card to assist in implementing the code. Among three pre-trained code models, CodeT5 can improve 4.86 on the BLEU metric and 3.05 on the CodeBLEU metric across the three datasets. The highest comprehensive improvement suggests that CodeT5 can be enhanced to a greater extent compared to the other models. As one of the most powerful pre-trained models for code generation [51], CodeT5 still possesses the potential for further advancements in code generation within the RAF.

Finding 1. RAF is universal for various existing pre-trained code models to improve the code generation performance on different datasets effectively. The utilization of retrieved code snippets in the framework can assist models in focusing on the semantic and structural information of generated code.

5.3 RQ2: Impact of Retrieval Techniques for RAF

In RQ1, our primary focus is to examine whether the RAF can improve the performance of various pre-trained code models. According to the comparison of the results in Table 3, the effectiveness and generalization of the RAF are validated. In this RQ, we assessed the ability of different retrieval techniques by comparing model performance before and after integration in the RAF for each of the three datasets separately, and the results are shown in Table 4.

The results obtained from the retrieval-augmented model for code generation, employing various retrieval techniques, merit further investigation. In the results of Table 4, all three models achieve the highest generation gains from the retrieved results of BM25 on CONCODE and HearthStone. Furthermore, CodeT5 achieves optimal performance when using the code snippets retrieved by BM25 on the CoNaLa dataset, with an improvement of 25.69% in the BLEU score and a 19.63% enhancement in the CodeBLEU score. These two improvements are both the highest in three pre-trained code models on three datasets. From this perspective, BM25, requiring no training, should be considered as the most promising retrieval technique to be explored within the RAF. Apart from BM25, leveraging the retrieved results of CoCoSoDa, CodeGen, and UniXcoder can enhance the performance of code generation maximally on CoNaLa, which indicates the effectiveness of CoCoSoDa for certain models and datasets.

Table 4. Results of the Retrieval-Augmented Model on Three Datasets with Different Retrieval Techniques

Model	Retrieval Technique	CONCODE		CoNaLa		HearthStone	
		BLEU	CodeBLEU	BLEU	CodeBLEU	BLEU	CodeBLEU
CodeGen	baseline	21.83	27.33	12.74	17.30	50.35	40.76
	BM25	24.99(14.48% ↑)	30.04(9.92% ↑)	15.58(22.29% ↑)	22.24(28.55% ↑)	54.35(7.94% ↑)	44.00(7.95% ↑)
	RetroMAE	20.14(7.74% ↓)	26.18(4.20% ↓)	12.93(1.49% ↑)	21.17(22.37% ↑)	9.40(81.33% ↓)	16.93(58.46% ↓)
	CodeBERT	24.18(10.77% ↑)	29.23(6.95% ↑)	13.61(6.83% ↑)	23.23(34.28% ↑)	52.08(3.44% ↑)	42.07(3.21% ↑)
	UniXcoder	24.76(13.42% ↑)	29.09(6.44% ↑)	14.63(14.84% ↑)	22.55(30.35% ↑)	51.44(2.16% ↑)	43.59(6.94% ↑)
	CoCoSoDa	24.98(14.43% ↑)	29.97(9.66% ↑)	16.50(29.51% ↑)	23.25(34.39% ↑)	47.27(6.12% ↓)	39.10(4.07% ↓)
UniXcoder	baseline	32.42	35.73	12.76	20.18	58.60	45.70
	BM25	35.80(10.43% ↑)	38.57(7.95% ↑)	13.69(7.29% ↑)	20.60(2.08% ↑)	61.00(4.10% ↑)	48.53(6.19% ↑)
	RetroMAE	32.02(1.23% ↓)	35.47(0.73% ↓)	13.77(7.92% ↑)	20.65(2.33% ↑)	55.89(4.62% ↓)	44.45(2.74% ↓)
	CodeBERT	35.15(8.42% ↑)	38.13(6.72% ↑)	13.61(6.67% ↑)	20.73(2.73% ↑)	56.12(4.23% ↓)	45.28(0.92% ↓)
	UniXcoder	34.79(7.31% ↑)	37.71(5.54% ↑)	13.32(4.39% ↑)	19.57(3.02% ↓)	55.79(4.80% ↓)	46.38(1.49% ↑)
	CoCoSoDa	34.05(5.03% ↑)	37.38(4.62% ↑)	14.55(14.03% ↑)	21.48(6.44% ↑)	59.25(1.11% ↑)	45.91(0.46% ↑)
CodeT5	baseline	36.93	40.15	10.55	18.80	55.96	44.49
	BM25	40.42(9.45% ↑)	43.48(8.29% ↑)	13.26(25.69% ↑)	22.49(19.63% ↑)	64.35(14.99% ↑)	49.80(11.94% ↑)
	RetroMAE	37.90(2.63% ↑)	42.30(5.35% ↑)	11.28(6.92% ↑)	21.76(15.74% ↑)	61.11(9.20% ↑)	48.32(8.61% ↑)
	CodeBERT	39.00(5.61% ↑)	43.05(7.22% ↑)	12.53(18.77% ↑)	21.18(12.66% ↑)	62.09(10.95% ↑)	49.09(10.34% ↑)
	UniXcoder	38.21(3.47% ↑)	41.38(3.06% ↑)	12.73(20.66% ↑)	21.03(11.86% ↑)	54.23(3.09% ↓)	49.19(10.56% ↑)
	CoCoSoDa	38.93(5.42% ↑)	42.72(6.40% ↑)	12.24(16.02% ↑)	21.77(15.80% ↑)	62.51(11.70% ↑)	48.87(9.84% ↑)

The percentages in parentheses following BLEU and CodeBLEU denote the enhancement achieved with various retrieval techniques compared to the baseline. Under each metric, the best performance is highlighted in bold.

A noteworthy observation in Table 4 is that the performance of CodeGen and UniXcoder declined by 7.74% and 1.32% in the BLEU score after incorporating the retrieved results from RetroMAE on CONCODE. Even though the performance of CodeT5 can improve to some extent, RetroMAE contributes the smallest improvement to model performance among the five retrieval techniques. The results indicate that the code snippets retrieved by RetroMAE contribute to a marginal and even inverse improvement of model performance. In other words, the retrieved results from RetroMAE are not always beneficial for the code generation process. Coincidentally, Table 4 shows a similar scenario about RetroMAE on HearthStone. As mentioned in Section 2.3, RetroMAE is employed to retrieve code directly without fine-tuning for code generation due to the absence of appropriate labeled natural language data. It is evident that RetroMAE performs subpar in this aspect on certain datasets, such as CONCODE and HearthStone, which feature distinct modes of expression compared to the field of NLP. Conversely, RetroMAE demonstrates relatively better performance in CoNaLa, where the natural language input aligns more closely with the text found in the realm of NLP. The substantial disparity between the code generation task and the text retrieval task could potentially explain why RetroMAE underperforms compared to the other four retrieval techniques.

According to the results in Table 4, CodeBERT and UniXcoder are not typically suggested as retrieval techniques in Retrieval Phase for the RAF. Code search models require fine-tuning on downstream datasets, which necessitates additional resources. While they outperform RetroMAE, they cannot attain a substantial improvement compared to BM25 and CoCoSoDa.

Finding 2. BM25 is suggested as a retrieval technique due to its consistently impressive performance and its inherent characteristic of requiring no training. As the state-of-the-art code search model, CoCoSoDa could play an important role in enhancing existing pre-trained models.

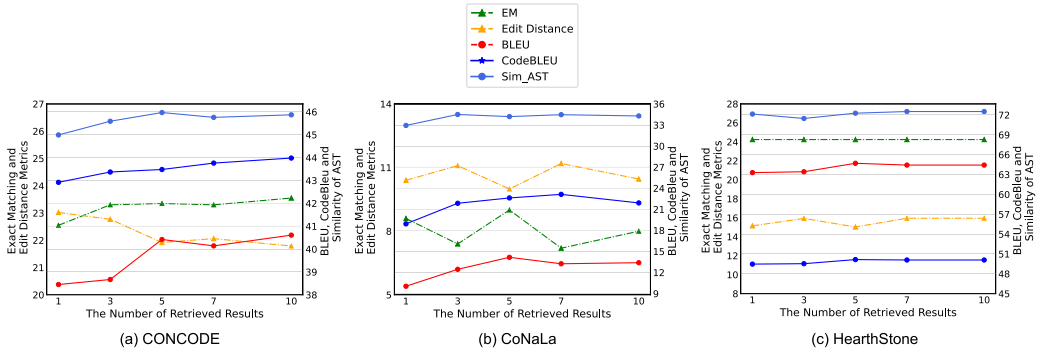


Fig. 2. The impact of the number of retrieved code snippets using SIF on the effectiveness of CodeT5. The green line and yellow line represent the EM metric and ED metric corresponding to the left vertical axis. The red, blue, and royal blue lines represent the BLEU metric, CodeBLEU metric, and similarity of AST, respectively. All the three metrics correspond to the right vertical axis.

5.4 RQ3: The Usage of the Retrieved Results for the Code Generation

The effectiveness and generalization of retrieval-augmented models for code generation has been proved in RQ1, and the impact of different retrieval techniques has been discussed in RQ2. In RQ3, we will further explore how to better utilize the retrieved results for code generation process. Initially, our investigation focuses on assessing the impact of the number of retrieved results on the final performance of the retrieval-augmented models using SIF, as employed in RQ1 and RQ2. Subsequently, several other fusion strategies are introduced to optimize the utilization of retrieved results, consequently enhancing the generation performance of the models. In detail, owing to the wonderful and consistent performance of CodeT5 across three datasets in RQ2, we opt to use CodeT5 as the base model for this RQ, which focuses on the utilization of retrieval results.

5.4.1 The Impact of the Number of Utilized Retrieved Results. To explore the impact of the number of retrieved code snippets on the model improvement, we conduct experiments with SIF, as employed in RQ1 and RQ2. We employ CodeT5 to concatenate 1, 3, 5, 7, and 10 retrieved code snippets across three datasets and visualized the experimental results in a line chart as Figure 2 shows.

As the number of retrieved results increases on CONCODE, an overall upward trajectory is observed in the performance of retrieval-augmented CodeT5 across all three metrics. At the same time, another important observation in Figure 2(a) is the inflection point of the line for BLEU metric. It occurs during the transition from concatenating three to concatenating five retrieval results. On CoNaLa, a comparable trend is noticeable in Figure 2(b), where the BLEU metric initially improves and then declines. Similar patterns are observed in other metrics as well, suggesting that as the number of code snippets increases, there is an initial enrichment of input information followed by a gradual diminishment. In other words, larger number of retrieved results does not indicate a better improvement. Considering that an increase in the length of input also leads to increased training costs, this inflection point could be a practical guideline for determining the optimal number of retrieved results. The metrics on HearthStone present a distinct scenario compared to the other two datasets: all three metrics remain unchanged when five results are concatenated. The reason is that the code snippets in HearthStone are all long, and over five code examples will be truncated by the model. This can also be seen as one of the main drawbacks of the SIF.

Table 5. Results of the Retrieval-Augmented Model for Three Datasets with Various Fusion Strategies

Dataset	Fusion Strategy	EM	BLEU	ED	Sim _{AST}	CodeBLEU	Training Costs	Inference Costs
CONCODE	-	21.45	36.93	23.12	46.64	40.15	128 min	547 s
	SIF	23.35	40.42	21.91	47.67	46.92	285 min	763 s
	VDF	11.30	28.72	22.55	45.65	39.37	393 min	1,662 s
	SEF	22.40	40.58	21.67	45.85	45.61	923 min	643 s
	SFF	21.90	40.84	19.10	46.45	46.40	917 min	805 s
CoNaLa	-	7.40	10.55	11.87	19.69	17.25	13 min	53 s
	SIF	8.00	13.26	10.46	23.29	21.46	18 min	81 s
	VDF	7.20	10.91	13.49	21.50	17.69	49 min	138 s
	SEF	1.80	15.91	13.92	10.78	17.95	47 min	63 s
	SFF	1.80	17.01	13.55	12.36	18.65	46 min	60 s
Hearthstone	-	19.70	55.96	19.76	55.14	44.49	50 min	47 s
	SIF	22.73	64.35	14.95	63.65	60.50	60 min	49 s
	VDF	22.73	64.78	16.09	65.59	60.42	320 min	77 s
	SEF	33.33	81.10	18.82	71.71	70.29	100 min	48 s
	SFF	34.85	81.89	19.71	73.00	71.76	107 min	57 s

“-” represents the base model fine-tuned on original datasets. Under each metric, the best performance is highlighted in bold. The training and inference costs represent the total training time and inference time for different fusion strategies separately.

5.4.2 Comparison of Various Fusion Strategy. In Section 5.4.1, the limitation of SIF is raised on HearthStone. In specific, the limitation is that only a finite number of retrieved results can be fused in the generation process of model, and the much longer augmented input will be truncated. In this case, more fusion strategies should be explored to enrich the RAF. According to the experiment results and findings in Section 5.4.1, we choose 5 as the number of retrieved code in the next experiments.

Table 5 shows the experiment results of the retrieval-augmented model for three datasets with four fusion strategies mentioned in Section 3.3. Across all three datasets, SIF and SFF consistently deliver the best performance. On CoNaLa, SIF outperforms SFF due to the short input and output lengths, which do not constrain CodeT5’s performance. Additionally, the data in CoNaLa lack a consistent structure, rendering SFF less effective as it struggles to extract a beneficial code sketch for model generation. Conversely, SFF excels on Hearthstone by capturing the analogous structure of similar code and effectively filtering out noisy variables. Another noteworthy observation is that the performance of SEF on Hearthstone is surpassed only by SFF and exceeds that of SIF. Moreover, the performance of SEF on the other two datasets closely rivals that of SFF. As outlined in Section 3.4, the code generation process using SFF is akin to that of SEF, with the sole distinction being whether it is the code or the code sketch that is appended after the natural language input. Consequently, SEF can be employed to enhance the model’s performance, particularly for small datasets such as Hearthstone, in a manner akin to data augmentation. Furthermore, SFF can build upon the improvements made by SEF to further enhance the model’s performance. VDF is also based on SEF. However, it does not bring additional enhancements to the model as SFF does. Specifically, VDF demonstrates decreased performance compared to SEF across three datasets. The potential reason for this could be twofold: either the encoder of CodeT5 falls short in capturing a robust representation of the augmented input, or the decoder of CodeT5 struggles to generate code adequately based on multiple vector representation.

In addition to evaluating the effects of various fusion strategies on model performance, Table 5 also provides insights into the training and inference costs for different fusion strategies on CodeT5. The training time is crucial to understand the computational overhead required by various fusion strategies. Across all three datasets, SIF emerges as the most time-efficient strategy

for training. Compared with SIF, VDF requires a longer duration, which is attributable to the more computational overhead associated with encoding. Since both SEF and SFF result in the training samples by a factor of k , they necessitate a training duration that is 2–7 times longer than that required for training directly on the original dataset. Regarding the inference costs, SEF solely concatenates the most similar code snippet behind the original input, resulting in the shortest input length and the quickest inference time. Overall, these fusion strategies exhibit comparable inference times, except for VDF. Considering both models' performance and costs, SIF is recommended as the fusion strategy in Fusion Phase.

Finding 3. The number of retrieved code snippets should be determined based on the attributes of specific datasets, such as input/output length. SIF and SFF can enhance existing pre-trained code models to a greater extent compared to other fusion strategies. SIF is the most recommended fusion strategy, balancing resource allocation and performance enhancement.

6 Discussion

6.1 Implications of Findings

6.1.1 Implications on the Effectiveness of the RAF. The effectiveness and generalization of the RAF have been proven to improve code generation performance for various existing pre-trained code models. This implies that it is a workable plan to improve the performance of the model by incorporating retrieval results that are similar to the input, without modifying the model architecture or size. Diverse models with different architectures can leverage this framework to produce more precise code snippets. In addition, the RAF can further enhance the specific model by picking out more powerful retrieval techniques and fusion strategies in the first two phases of this framework.

To further investigate the effectiveness of the RAF for LLMs, we additionally conduct experiments using three popular LLMs: ChatGLM3-6B [15], CodeLlama-7B [59], and DeepSeek-Coder-6.7B [23]. We also explore the impact of various retrieval techniques for code generation when utilizing these LLMs. The experiment results of the LLMs with RAF on three datasets are shown in Table 6. The retrieved similar code snippets, as references during the code generation process, are integrated into the inputs of the LLMs via Prompt Engineering. The prompts are constructed following [43]. More detailed information about the experiment can be found in our code repository.

As shown in Table 6, all LLMs improve their performance across all three datasets with the RAF. On CONCODE, the BLEU metric ratio, resulting from prompting ChatGLM with the similar codes retrieved by CoCoSoDa, compared to using ChatGLM directly, stands at 98.33. On Hearthstone, the BLEU metric ratio, when using BM25 compared to the original ChatGLM, even escalates to 198.67. These substantial improvements demonstrate that the RAF can effectively enhance the performance of the LLMs in generating target code during the inference phase by providing similar code. This suggests that integrating similar code snippets can greatly aid in the code generation process, leading to more accurate and efficient generation results.

6.1.2 Implications on the Utilization of RAF. Our experiment results demonstrate that different retrieval techniques in Retrieval Phase influence the performance of retrieval-augmented models for code generation. As shown in Tables 4 and 6, among five retrieval techniques, BM25 and CoCoSoDa perform best with both LLMs and pre-trained models. To facilitate a clear comparison of the costs associated with various retrieval techniques, we divide the retrieval process costs into two parts:

Table 6. Results of LLMs with the RAF on Three Datasets with Different Retrieval Techniques

Model	Retrieval Technique	CONCODE		CoNaLa		HearthStone	
		BLEU	CodeBLEU	BLEU	CodeBLEU	BLEU	CodeBLEU
ChatGLM	baseline	0.06	20.21	0.31	13.43	0.03	8.05
	BM25	5.76(96.00)	32.17(1.59)	0.70(2.26)	21.39(1.59)	5.96(198.67)	22.19(2.76)
	RetroMAE	2.55(42.50)	26.86(1.33)	0.79(2.55)	21.92(1.63)	3.44(114.67)	14.72 (1.83)
	CodeBERT	5.89(98.17)	31.02(1.53)	0.84(2.71)	22.36(1.66)	5.35(178.33)	19.59(2.43)
	UniXcoder	5.60(93.33)	31.30(1.55)	0.89(2.87)	20.90(1.56)	5.31(177.00)	19.47(2.42)
	CoCoSoDa	5.90(98.33)	32.33(1.60)	0.95(3.06)	24.16(1.80)	5.11(170.33)	13.74(1.71)
CodeLlama	baseline	0.62	19.10	0.04	14.59	0.05	11.63
	BM25	7.45(12.02)	38.35(2.01)	0.64(16.00)	23.88(1.64)	7.35(147.00)	47.69(4.10)
	RetroMAE	3.72(6.00)	31.43(1.65)	0.77(19.25)	24.02(1.65)	6.54(130.80)	42.71(3.67)
	CodeBERT	7.52(12.13)	38.70(2.03)	1.11(27.75)	25.15(1.72)	7.03(140.6)	43.65(3.75)
	UniXcoder	6.84(11.03)	38.46(2.01)	1.11(27.75)	24.86(1.70)	6.07(121.40)	38.94(3.35)
	CoCoSoDa	7.54(12.16)	38.86(2.03)	1.25(31.25)	25.93(1.78)	6.36(127.20)	41.43(3.56)
DeepSeek-Coder	baseline	0.15	23.31	0.17	16.46	0.06	11.08
	BM25	4.88(32.53)	37.64(1.61)	1.09(6.41)	25.57(1.55)	5.44(90.67)	46.21(4.17)
	RetroMAE	2.22(14.80)	32.82(1.41)	0.95(5.59)	25.74(1.56)	4.41(73.50)	41.39(3.74)
	CodeBERT	5.07(33.80)	37.89(1.63)	1.12(6.59)	25.09(1.52)	4.64(77.33)	41.26(3.72)
	UniXcoder	4.68(31.20)	38.27(1.64)	1.01(5.94)	24.15(1.47)	4.33(72.17)	39.45(3.52)
	CoCoSoDa	5.45(36.33)	38.75(1.66)	1.22(7.18)	25.78(1.57)	4.94(82.33)	41.18(3.72)

The values in parentheses following BLEU and CodeBLEU indicate the ratio of the performance with the RAF compared to the baseline. Variable categories are the same with Table 4.

training cost and retrieval cost. The experimental results are presented in Table 7. For text retrieval algorithms, there is no training cost as no training is required. The total cost is calculated by summing the training cost (if applicable) and the retrieval cost. BM25 is executed on a CPU, while all other deep learning-based retrieval technologies are run on a single A100 GPU to calculate the costs. The total costs indicate that in our experiments, BM25 incurs a smaller cost than the deep learning-based code search models while achieving superior results. However, the advantage of BM25 diminishes as the search volume and input length increases, as evidenced in CONCODE. According to Table 1, the input length of CONCODE is the longest, and the size of the CONCODE training set is the largest among the three datasets. Since BM25 retrieves code by calculating the similarity between natural languages at the token level, its cost on CONCODE is higher than two other datasets. The total costs of BM25 are comparable to deep learning-based retrieval techniques (including training and retrieval). Retrieving similar code snippets for just one instance takes about 4 s (199.25 s/50), which is intolerable in a real-world code generation scenario. Therefore, BM25 is recommended for datasets with short inputs or small sizes. However, for datasets with longer input/output lengths, more efficient code search models could be crucial. Moreover, the experiment results of the LLMs highlight the potential of CoCoSoDa within the RAF, suggesting that future work should delve deeper into exploring more code search models.

Among various fusion strategies, the SIF strategy proves to be a simple, direct, and effective fusion strategy. For the dataset with a clear structure, SFF has the capacity to improve the model performance, as an optimal fusion strategy. However, this optimal performance comes at a cost: the fusion process of SFF necessitates training the model for k times the sample size. Our experiment results show that their training time is 2–7 times longer than fine-tuning directly on the original dataset. Therefore, when considering both the trade-off between the performance of the fusion strategies and the training costs, SIF is the most recommended.

There are two considerations when using SIF: the number and the order of concatenated retrieved code. Determining an appropriate number of concatenated retrieved code involves the capacity of

Table 7. The Training Costs Per Epoch and Average Retrieval Costs Per 50 Instances of Different Retrieval Techniques on Different Datasets

Dataset	Retrieval Techniques	Training Cost	Retrieval Cost	Total Costs
CONCODE	BM25	-	199.25 s	7,970.00 s
	RetroMAE	-	2.02 s	81.00 s
	CodeBERT	1,089.71 s	14.35 s	6,166.05 s
	UniXcoder	1,680.13 s	12.50 s	8,900.13 s
	CoCoSoDa	2,001.02 s	1.23 s	10,051.02 s
CoNaLa	BM25	-	1.09 s	109.00 s
	RetroMAE	-	4.00 s	40.00 s
	CodeBERT	41.13 s	2.90 s	234.65 s
	UniXcoder	42.00 s	7.67 s	809.00 s
	CoCoSoDa	47.00 s	1.03 s	150.00 s
HearthStone	BM25	-	1.05 s	1.39 s
	RetroMAE	-	1.52 s	2.00 s
	CodeBERT	32.40 s	5.97 s	169.88 s
	UniXcoder	20.38 s	15.15 s	115.53 s
	CoCoSoDa	49.47 s	3.79 s	249.26 s

The retrieved codebases refer to the training set of the corresponding dataset. “-” represents that the retrieval technique does not need to be trained.

Table 8. Results of the Retrieval-Augmented CodeT5 for Three Datasets with Different Sample Ordering

Dataset	Sample Ordering Strategy	EM	BLEU	ED	Sim _{AST}	CodeBLEU
CONCODE	Ascending Ordering	23.35	40.42	21.91	47.67	46.92
	Descending Ordering	22.55	36.82	24.23	47.02	45.73
CoNaLa	Ascending Ordering	8.00	13.26	10.46	23.29	21.46
	Descending Ordering	7.20	12.18	11.32	22.84	20.43
Hearthstone	Ascending Ordering	22.73	64.35	14.95	63.65	60.50
	Descending Ordering	21.21	64.25	15.48	62.81	60.24

The retrieval technique is BM25, and the fusion strategy is SIF. Under each metric, the best performance is highlighted in bold.

the models and the attributes of specific datasets. It is recommended to opt for a median number based on the length of data in different datasets. We also conduct experiments with the retrieved code snippets input in ascending ordering (sorted from high to low based on similarity) and in descending ordering (sorted from low to high based on similarity). As indicated in Table 8, the results demonstrate that the ascending ordering outperforms the descending ordering.

6.2 Case Study

In this section, we provide two case studies to qualitatively compare the base model with the retrieval-augmented model and the retrieved results from different retrieval techniques.

6.2.1 The Effectiveness of the RAF for Code Generation. Figure 3 shows a case including the original data, the output of original CodeT5, the retrieved results from BM25, and the output of

```

getinstance method of logoutrequestsender, as it is a singleton
concode_field_sep LogoutRequestSender instance
concode_elem_sep Log log
concode_elem_sep ExecutorService threadPool
concode_elem_sep SingleLogoutRequestDT0 logoutReqDT0
concode_field_sep void sendLogoutRequests
concode_elem_sep void run
concode_elem_sep int derivePortFromAssertionConsumerURL

(a) NL Input
LogoutRequestSender function(){
    if(instance == null){
        synchronized(LogoutRequestSender.class){
            if(instance==null){
                instance = new LogoutRequestSender();
            }
        }
    }
    return instance;
}

(c) The Output of CodeT5
LogoutRequestSender function(){
    return instance;
}

(e) The Output of Retrieval-Augmented CodeT5
LogoutRequestSender function(){
    return instance;
}

LogoutRequestSender function(){
    return instance;
}

(b) Ground Truth
retrieved_code1:
    NamespaceTranslatorFactory function(){
        return instance;
    }
retrieved_code2:
    GpmDoubleFormatter function(){
        return INSTANCE;
    }
retrieved_code3:
    GpmStringFormatter function(){
        return INSTANCE;
    }
retrieved_code4:
    GpmIntegerFormatter function(){
        return INSTANCE;
    }
retrieved_code5:
    void function(){
        System.out.println("srini_string");
        DownloadManager loc0 = DownloadManager.getInstance();
        assertNotNull(loc0);
    }

(d) Retrieved Results

```

Fig. 3. Case study on CONCODE with RAF, where the retrieval technique is BM25, fusion strategy is SIF, and the pre-trained code model is CodeT5.

```

get filename without extension from filename os.path.splitext(filename)[0]

(a) NL Input
Extracting extension from filename in Python

(b) Ground Truth
How to get the name of an open file?

(c-1) Similar NL input retrieved by BM25
filename, file_extension = os.path.splitext('/path/to/somefile.ext')

(e-1) Similar NL input retrieved by RetroMAE
filename.name

(c-2) The corresponding code snippet of BM25
'.'.join(filename.split('.')[::-1]) if '.' in filename else filename

(e-2) The corresponding code snippet of RetroMAE
os.path.basename(f.name)

(f) Similar code snippet retrieved by CodeBERT
filename = filename[::-1]

(d) Similar code snippet retrieved by CoCoSoDa
(g) Similar code snippet retrieved by UniXcoder

```

Fig. 4. The most similar code snippet retrieved by different retrieval techniques on CoNaLa.

retrieval-augmented CodeT5 on CONCODE. The requirement is “getinstance methods of logoutrequestsender, as it is a singleton,” and the other environment information is shown in Figure 3(a). In Figure 3(c), we can observe that the code generated by the original CodeT5 is more complex but does not adhere to the intended specifications. Indeed, the retrieved results (d) obtained from BM25 exhibit similarity to the ground truth (b). This similarity serves as a valuable reference point for CodeT5, aiding in understanding the genuine intent and determining the appropriate results to be returned. In this way, retrieval-augmented CodeT5 can generate the same code (e) as ground truth (b).

6.2.2 Analysis on the Retrieved Results by Different Retrieval Techniques. As shown in Figure 4, different retrieval techniques in the RAF identify various similar code snippets for the subsequent fusion and generation phases. Text retrieval algorithms, such as BM25 and RetroMAE, compute the similarity between the natural language input and natural language description in the retrieved codebase. The corresponding code snippets are then returned based on the retrieved natural language description. Due to the underlying text matching principle, the retrieved natural language description shares many keywords with the input, such as “extension” and “filename” (as shown in Figure 4(a) and (c-1)). The similar natural language description facilitates the return of similar code snippets related to the ground truth. RetroMAE retrieves natural language descriptions by calculating their semantic similarity. As shown in Figure 4(e-1), the retrieved natural language description conveys the semantics of obtaining the file name but ignores the condition “without

extension,” leading to poor retrieval performance. The other three code search models retrieve similar code snippets by directly calculating the similarity between natural language and code snippets. In this case, CoCoSoDa (Figure 4(d)) and CodeBERT (Figure 4(f)) retrieve code that meets the requirements of the descriptions, while UniXcoder (Figure 4(g)) does not fully understand the descriptions and returns an intermediate result close to the target answer of CoCoSoDa.

6.3 Future Work

Based on our findings and implications, in this section, we present two possible future works for retrieval-augmented code generation.

6.3.1 Active Retrieval. Our experimental results in Section 5.3 show that not all retrieved code snippets are beneficial for the final generated output. If the retrieved code is irrelevant to the input natural language description, it may confuse code generation models as noise, reducing the generation performance. Additionally, as shown in Table 7, retrieval incurs extra costs compared to using the code generation model directly. Thus, determining whether retrieval should be performed to improve model performance requires further investigation, which is called active retrieval. Considering that active retrieval improves efficiency and usability in NLP tasks [2, 10, 35] and code-related tasks [70], we believe it can also optimize and inform the future design of RAF for code generation tasks.

6.3.2 Retrieval Database Construction. In our experiments and previous works, the RAF for code generation tasks regard the training set as the retrieval database. However, with the development of general models, evaluation datasets often exclude training sets (e.g., HumanEval [9] and MBPP [3]). Therefore, constructing a comprehensive, diverse, and informative retrieval database is essential for applying the RAF to various code generation scenarios. Moreover, based on the database, exploring the potential of fine-tuning deep learning-based models on code generation datasets is a worthwhile endeavor.

6.4 Threats to Validity

The Generalization of Model Results. Code generation stands as one of the most pivotal tasks in code intelligence, where code generation models persistently undergo innovation and evolution. Given the absence of empirical analysis for retrieval-augmented code generation models, it becomes crucial to conscientiously design a considerable number of experiments to address this research gap. We select three popular pre-trained models with different architectures for a constrained experimental exploration. We have made diligent efforts to summarize the experimental results, leading to the discovery of several compelling findings. Nevertheless, there remains uncertainty regarding whether these findings remain applicable to larger models or models with differing architectures.

The Replication of Our Experiments. In this article, we perform diverse retrieval techniques along with training and testing of the augmented models. Nevertheless, it is essential to note that both the code search models and the pre-trained code models based on deep learning could be influenced by various factors, including different devices and parameter settings. To address this issue, we have made our retrieval-augmented datasets³ and code repository⁴ publicly available. By utilizing our retrieval results directly, along with the code and parameter settings from previous work, the consistency of the code generation tasks with our experiment results will be ensured to some extent.

³https://drive.google.com/drive/folders/1G_ssf9gCX38Yb7FAjsllRxiYPO44omvP?usp=drive_link

⁴<https://github.com/watreyoung/RACG>

Limited Dataset. The experiment results are based on three datasets for code generation task. While we have selected the most widely used and representative datasets for our experiment, there remains a distinct gap between the data within these datasets and the real, specific development environment context. For example, CONCODE is the most used benchmark for code generation task, but the pre-processing makes human hard to understand the code intuitively. In short, it is difficult for human developers to write code the same as the ground truth according to the natural language description on CONCODE. This issue persists across numerous tasks within code intelligence, indicating a potential future research direction to construct a dataset that better mirrors real-world development scenarios.

7 Conclusion

In this article, we experimentally investigate the effectiveness and generalization of the RAF for code generation on three different datasets. Our study shows that RAF can indeed improve the code generation performance of existing code pre-trained models, such as CodeGen, UniXcoder, and CodeT5. Besides, we explore the impact of different retrieval techniques and different fusion strategies on RAF. Plenty of experimental results are listed and discussed in Section 5. We summarize our findings and provide some implications for the utilization of RAF for code generation. These insights may assist researchers in leveraging the RAF to enhance their own models for code generation.

References

- [1] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT '21)*. Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tür, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.), Association for Computational Linguistics, 2655–2668.
- [2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *Proceedings of the 12th International Conference on Learning Representations (ICLR '24)*. OpenReview.net.
- [3] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, et al. 2021. Program synthesis with large language models. arXiv:2108.07732. Retrieved from <https://arxiv.org/abs/2108.07732>
- [4] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. 2023. CodePlan: Repository-level coding using LLMs and planning. arXiv:2309.12499. Retrieved from <https://arxiv.org/abs/2309.12499>
- [5] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q. Feldman, et al. 2022. A scalable and extensible approach to benchmarking NL2Code for 18 programming languages. arXiv:2208.08227. Retrieved from <https://arxiv.org/abs/2208.08227>
- [6] ChatGPT. 2022. ChatGPT. Retrieved from <https://openai.com/blog/chatgpt>
- [7] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT5: Code generation with generated tests. In *Proceedings of the 11th International Conference on Learning Representations*.
- [8] Danqi Chen, Adam Fisch, Jason Weston, and Antoine Bordes. 2017. Reading Wikipedia to answer open-domain questions. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL '17), Volume 1: Long Papers*. Regina Barzilay and Min-Yen Kan (Eds.), Association for Computational Linguistics, 1870–1879.
- [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from <https://arxiv.org/abs/2107.03374>
- [10] Qinyuan Cheng, Xiaonan Li, Shimin Li, Qin Zhu, Zhangyue Yin, Yunfan Shao, Linyang Li, Tianxiang Sun, Hang Yan, and Xipeng Qiu. 2024. Unified active retrieval for retrieval augmented generation. arXiv:2406.12534. Retrieved from <https://arxiv.org/abs/2406.12534>
- [11] Fenia Christopoulou, Gerasimos Lampouras, Milan Gritta, Guchun Zhang, Yinpeng Guo, Zhongqi Li, Qi Zhang, Meng Xiao, Bo Shen, Lin Li, et al. 2022. PanGu-Coder: Program synthesis with function-level language modeling. arXiv:2207.11280. Retrieved from <https://arxiv.org/abs/2207.11280>

- [12] CodeGeeX. 2022. CodeGeeX. Retrieved from <https://models.aminer.cn/codegeex/blog/>
- [13] Deborah A. Dahl, Madeleine Bates, Michael Brown, William M. Fisher, Kate Hunicke-Smith, David S. Pallett, Christine Pao, Alexander I. Rudnicky, and Elizabeth Shriberg. 1994. Expanding the scope of the ATIS task: The ATIS-3 corpus. In *Proceedings of the Workshop on Human Language Technology*. Morgan Kaufmann.
- [14] Dawn Drain, Changran Hu, Chen Wu, Mikhail Breslav, and Neel Sundaresan. 2021. Generating code with the help of retrieved template functions and stack overflow answers. arXiv:2104.05310. Retrieved from <https://arxiv.org/abs/2104.05310>
- [15] Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, and Jie Tang. 2022. GLM: General language model pretraining with autoregressive blank infilling. In *Proceedings of the Association for Computational Linguistics (ACL)*. Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.), Association for Computational Linguistics, 320–335.
- [16] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: Finding (EMNLP '20)*. Trevor Cohn, Yulan He, and Yang Liu (Eds.), Association for Computational Linguistics, 1536–1547.
- [17] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A generative model for code infilling and synthesis. In *Proceedings of the 11th International Conference on Learning Representations*.
- [18] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. 2020. The Pile: An 800GB dataset of diverse text for language modeling. arXiv:2101.00027. Retrieved from <https://arxiv.org/abs/2101.00027>
- [19] Shuzheng Gao, Xin-Cheng Wen, Cuiyun Gao, Wenxuan Wang, Hongyu Zhang, and Michael R. Lyu. 2023. What makes good in-context demonstrations for code intelligence tasks with LLMs? In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 761–773.
- [20] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, et al. 2023. Retrieval-augmented generation for large language models: A survey. arXiv:2312.10997. Retrieved from <https://arxiv.org/abs/2312.10997>
- [21] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (ACL '22)*. Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.), Association for Computational Linguistics, 7212–7225.
- [22] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2021. GraphCodeBERT: Pre-training code representations with data flow. In *Proceedings of the 9th International Conference on Learning Representations (ICLR '21), Virtual Event*. OpenReview.net.
- [23] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, et al. 2024. DeepSeek-Coder: When the large language model meets programming—The rise of code intelligence. arXiv:2401.14196. Retrieved from <https://arxiv.org/abs/2401.14196>
- [24] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. Retrieval augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning (ICML '20), Virtual Event (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 3929–3938.
- [25] Tatsunori B. Hashimoto, Kelvin Guu, Yonatan Oren, and Percy Liang. 2018. A retrieve-and-edit framework for predicting structured outputs. In *Proceedings of the Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018 (NeurIPS '18)*. Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett (Eds.), 10073–10083.
- [26] Shirley Anugrah Hayati, Raphaël Olivier, Pravalika Avvaru, Pengcheng Yin, Anthony Tomasic, and Graham Neubig. 2018. Retrieval-based neural code generation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.), Association for Computational Linguistics, 925–930.
- [27] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross B. Girshick. 2020. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR '20)*. IEEE, 9726–9735.
- [28] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. 2021. Measuring coding challenge competence with APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021*. Joaquin Vanschoren and Sai-Kit Yeung (Eds.), MIT Press.

- [29] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2023. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. arXiv:2311.05232. Retrieved from <https://arxiv.org/abs/2311.05232>
- [30] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet challenge: Evaluating the state of semantic code search. arXiv:1909.09436. Retrieved from <https://arxiv.org/abs/1909.09436>
- [31] Srinivasan Iyer, Alvin Cheung, and Luke Zettlemoyer. 2019. Learning programmatic idioms for scalable semantic parsing. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP '19)*. Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.), Association for Computational Linguistics, 5425–5434.
- [32] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2018. Mapping language to code in programmatic context. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.), Association for Computational Linguistics, 1643–1652.
- [33] Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main (EACL '21)*. Paola Merlo, Jörg Tiedemann, and Reut Tsarfaty (Eds.), Association for Computational Linguistics, 874–880.
- [34] Xue Jiang, Yihong Dong, Lecheng Wang, Qiwei Shang, and Ge Li. 2023. Self-planning code generation with large language model. arXiv:2303.06689. Retrieved from <https://arxiv.org/abs/2303.06689>
- [35] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP '23)*. Houda Bouamor, Juan Pino, and Kalika Bali (Eds.), Association for Computational Linguistics, 7969–7992.
- [36] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. 2020. Generalization through memorization: Nearest neighbor language models. In *Proceedings of the 8th International Conference on Learning Representations (ICLR '20)*. OpenReview.net.
- [37] Philippe Laban, Wojciech Kryscinski, Divyansh Agarwal, Alexander R. Fabbri, Caiming Xiong, Shafiq Joty, and Chien-Sheng Wu. 2023. SummEdits: Measuring LLM ability at factual reasoning through the lens of summarization. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP '23)*. Houda Bouamor, Juan Pino, and Kalika Bali (Eds.), Association for Computational Linguistics, 9662–9676.
- [38] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Hoi. 2022. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Proceedings of the Advances in Neural Information Processing Systems*. Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho (Eds.).
- [39] Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020 (NeurIPS '20)*, Virtual. Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.), MIT Press.
- [40] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2023. Enabling programming thinking in large language models toward code generation. arXiv:2305.06599. Retrieved from <https://arxiv.org/abs/2305.06599>
- [41] Jia Li, Yongmin Li, Ge Li, Zhi Jin, Yiyang Hao, and Xing Hu. 2023. SkCoder: A sketch-based approach for automatic code generation. arXiv:2302.06144. Retrieved from <https://arxiv.org/abs/2302.06144>
- [42] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2023. Towards enhancing in-context learning for code generation. arXiv:2303.17780. Retrieved from <https://arxiv.org/abs/2303.17780>
- [43] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2024. AceCoder: An effective prompting technique specialized in code generation. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–26.
- [44] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [45] Wang Ling, Phil Blunsom, Edward Grefenstette, Karl Moritz Hermann, Tomás Kociský, Fumin Wang, and Andrew W. Senior. 2016. Latent predictor networks for code generation. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL 2016)*, Volume 1: Long Papers. The Association for Computer Linguistics.
- [46] Chao Liu, Xin Xia, David Lo, Cuiyun Gao, Xiaohu Yang, and John C. Grundy. 2022. Opportunities and challenges in code search tools. *ACM Computer Surveys* 54, 9 (2022), 196:1–196:40.
- [47] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. arXiv:1907.11692. Retrieved from <https://arxiv.org/abs/1907.11692>

- [48] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, Virtual*. Joaquin Vanschoren and Sai-Kit Yeung (Eds.), MIT Press.
- [49] Jianmo Ni, Chenguang Zhu, Weizhu Chen, and Julian J. McAuley. 2019. Learning to attend on essential terms: An enhanced retriever-reader model for open-domain question answering. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT '19), Volume 1 (Long and Short Papers)*. Jill Burstein, Christy Doran, and Tamar Solorio (Eds.), Association for Computational Linguistics, 335–344.
- [50] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An open large language model for code with multi-turn program synthesis. In *Proceedings of the 11th International Conference on Learning Representations*.
- [51] Changan Niu, Chuanyi Li, Vincent Ng, Dongxiao Chen, Jidong Ge, and Bin Luo. 2023. An empirical comparison of pre-trained models of source code. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE '23)*. IEEE, 2136–2148.
- [52] OpenAI. 2023. GPT-4 technical report. arXiv:2303.08774. Retrieved from <https://arxiv.org/abs/2303.08774>
- [53] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. ACL, 311–318.
- [54] Md. Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval augmented code generation and summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2021, Virtual Event*. Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.), Association for Computational Linguistics, 2719–2734.
- [55] Maxim Rabinovich, Mitchell Stern, and Dan Klein. 2017. Abstract syntax networks for code generation and semantic parsing. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL '17), Volume 1: Long Papers*. Regina Barzilay and Min-Yen Kan (Eds.), Association for Computational Linguistics, 1139–1149.
- [56] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI Blog* 1, 8 (2019), 9.
- [57] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research* 21 (2020), 140:1–140:67.
- [58] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: A method for automatic evaluation of code synthesis. arXiv:2009.10297. Retrieved from <https://arxiv.org/abs/2009.10297>
- [59] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code Llama: Open foundation models for code. arXiv:2308.12950. Retrieved from <https://arxiv.org/abs/2308.12950>
- [60] Ensheng Shi, Yanlin Wang, Wenchao Gu, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Hongbin Sun. 2022. CoCoSoDa: Effective contrastive learning for code Search. arXiv:2204.03293. Retrieved from <https://arxiv.org/abs/2204.03293>
- [61] Zeyu Sun, Qihao Zhu, Yingfei Xiong, Yican Sun, Lili Mou, and Lu Zhang. 2020. TreeGen: A tree-based transformer architecture for code generation. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence (AAAI '20), the 32nd Innovative Applications of Artificial Intelligence Conference (IAAI '20), the 10th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI '20)*. AAAI Press, 8984–8991.
- [62] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. IntelliCode compose: Code generation using transformer. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*. Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.), ACM, 1433–1443.
- [63] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. arXiv:2104.08663. Retrieved from <https://arxiv.org/abs/2104.08663>
- [64] Andrew Trotman, Antti Puurula, and Blake Burgess. 2014. Improvements to BM25 and language models examined. In *Proceedings of the 2014 Australasian Document Computing Symposium (ADCS '14)*. J. Shane Culpepper, Laurence Anthony F. Park, and Guido Zucco (Eds.), ACM, 58–65.
- [65] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 30 (2017).

- [66] Xin Wang, Yasheng Wang, Yao Wan, Fei Mi, Yitong Li, Pingyi Zhou, Jin Liu, Hao Wu, Xin Jiang, and Qun Liu. 2022. Compilable neural code generation with compiler feedback. In *Findings of the Association for Computational Linguistics (ACL '22)*. Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.), Association for Computational Linguistics, 9–19.
- [67] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. CodeT5+: Open code large language models for code understanding and generation. arXiv:2305.07922. Retrieved from <https://arxiv.org/abs/2305.07922>
- [68] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP 2021), Virtual Event*. Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.), Association for Computational Linguistics, 8696–8708.
- [69] Zhiruo Wang, Grace Cuenca, Shuyan Zhou, Frank F. Xu, and Graham Neubig. 2023. MCoNaLa: A benchmark for code generation from multiple natural languages. In *Findings of the Association for Computational Linguistics (EACL '23)*. Andreas Vlachos and Isabelle Augenstein (Eds.), Association for Computational Linguistics, 265–273.
- [70] Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. 2024. Repoformer: Selective retrieval for repository-level code completion. arXiv:2403.10059. Retrieved from <https://arxiv.org/abs/2403.10059>
- [71] Shitao Xiao, Zheng Liu, Yingxia Shao, and Zhao Cao. 2022. RetroMAE: Pre-training retrieval-oriented language models via masked auto-encoder. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP '22)*. Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.), Association for Computational Linguistics, 538–548.
- [72] Pengcheng Yin, Bowen Deng, Edgar Chen, Bogdan Vasilescu, and Graham Neubig. 2018. Learning to mine aligned code and natural language pairs from stack overflow. In *Proceedings of the 15th International Conference on Mining Software Repositories (MSR '18)*. Andy Zaidman, Yasutaka Kamei, and Emily Hill (Eds.), ACM, 476–486.
- [73] Pengcheng Yin and Graham Neubig. 2017. A syntactic neural model for general-purpose code generation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL '17), Volume 1: Long Papers*. Regina Barzilay and Min-Yen Kan (Eds.), Association for Computational Linguistics, 440–450.
- [74] Pengcheng Yin and Graham Neubig. 2018. TRANX: A transition-based neural abstract syntax parser for semantic parsing and code generation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP '18): System Demonstrations*. Eduardo Blanco and Wei Lu (Eds.), Association for Computational Linguistics, 7–12.
- [75] John M. Zelle and Raymond J. Mooney. 1996. Learning to parse database queries using inductive logic programming. In *Proceedings of the 13th National Conference on Artificial Intelligence and 8th Innovative Applications of Artificial Intelligence Conference (AAAI '96, IAAI '96)*. William J. Clancey and Daniel S. Weld (Eds.), AAAI Press/The MIT Press, 1050–1055.
- [76] Wayne Xin Zhao, Jing Liu, Ruiyang Ren, and Ji-Rong Wen. 2024. Dense text retrieval based on pretrained language models: A survey. *ACM Transactions on Information Systems* 42, 4 (2024), 89:1–89:60.
- [77] Shuyan Zhou, Uri Alon, Frank F. Xu, Zhengbao Jiang, and Graham Neubig. 2023. DocPrompting: Generating code by retrieving the docs. In *Proceedings of the 11th International Conference on Learning Representations (ICLR '23)*. OpenReview.net.
- [78] Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chandan K. Reddy. 2022. XLCoS: A benchmark dataset for cross-lingual code intelligence. arXiv:2206.08474. Retrieved from <https://arxiv.org/abs/2206.08474>

Received 18 November 2023; revised 9 October 2024; accepted 1 January 2025